

Lab Exercise 11: Binary Search on Monotonic Answer Spaces (Aggressive Cows variant)

Objective: To write an optimization program using advanced Binary Search to find the maximum possible minimum distance between allocated elements.

Aim: To solve resource allocation problems over continuous search spaces in $O(n \log \text{max_dist})$ runtime instead of checking all options iteratively.

Theory: When an option matrix is monotonic (meaning if a distance SD is valid, all distances less than SD are also valid), a binary search can find the optimal value. The algorithm checks the midpoint position value using a helper validation function, quickly cutting the search space in half.

Algorithm / Procedure:

1. Sort the input coordinate array stalls. Set the search space boundaries: low = 1 and high = stalls[n-1] - stalls[0].
2. Compute mid = low + (high - low) / 2.
3. Use a helper function isAllocationPossible(stalls, mid, targetCount) to check if the current distance allows all items to be placed safely.
4. If valid, log the configuration as a possible solution and shift the low boundary up (low = mid + 1) to find a larger minimum distance. If invalid, narrow the search space down (high = mid - 1).

Program Code: Problem7_AggressiveCows.java

```
import java.util.*;
```

```
public class Problem7_AggressiveCows {
```

```
    private static boolean isAllocationPossible(int[] stalls, int minDist, int cows) {
```

```
        // TODO: Implement checking logic to place items given a minimum distance constraint
```

```
        return false; // placeholder
```

```
    }
```

```
    public static int findOptimalAllocationDistance(int[] stalls, int cows) {
```

```
        Arrays.sort(stalls);
```

```

// TODO: Implement binary search over answer space loop here

return 0; // placeholder
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of stalls and cows: ");

    int n = sc.nextInt(); int cows = sc.nextInt();

    int[] stalls = new int[n];

    for(int i=0; i<n; i++) stalls[i] = sc.nextInt();


    int result = findOptimalAllocationDistance(stalls, cows);

    System.out.println("Maximum possible minimum distance: " + result);

    sc.close();

}
}

```

Sample Input / Output:

Input:

5 3

1 2 8 4 9

Output:

Maximum possible minimum distance: 3

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Correct implementation of the binary search structure over answers (3 Marks)
- Accurate logic inside the distance checking function (2 Marks)
- Correct boundary updates and sorting setup (2 Marks)
- Code readability and structure (1 Mark)